
修饰符 运算符重载等

复习

- | | |
|---------------------|----|
| 1. 面向对象和面向过程的区别 | 了解 |
| 2. 类与对象的基本概念 | 了解 |
| 3. 类class与类对象 | 熟悉 |
| 4. 构造函数与析构函数 | 重点 |
| 5. 动态创建对象new/delete | 重点 |
| 6. 动态创建对象数组 | 重点 |
| 7. 成员变量与成员函数 | 重点 |
| 8. 静态成员变量与静态成员函数 | 重点 |
| 9. const修饰类的成员函数 | 熟悉 |
| 10. this指针 | 重点 |

目录

- | | |
|-------------|----|
| 1. 修饰符 | 重点 |
| 2. 友元类与友元函数 | 重点 |
| 3. 运算符重载 | 重点 |

访问限定符

在 C++ 中，所有的 成员变量 和 成员函数 都有访问权限，所谓的访问权限，就是到底能不能访问该类中的成员变量和成员函数。

C++ 中，用来控制访问权限的 关键字 有 `public`、`protected` 和 `private`，它们分别表示公有的、受保护的和私有的，同时，它们被统称为成员访问限定符。

访问限定符

public

可以被该类中的函数、子类的函数、其友元函数访问，也可以由该类的对象访问。

protected

可以被该类中的函数、子类的函数、以及其友元函数访问，但不能被该类的对象访问。

private

只能由该类中的函数、其友元函数访问，不能被任何其他访问，**该类的对象也不能访问**。

访问限定符

```
class Student{
public:
    int v1=100;
private:
    int v2=200;
protected:
    int v3=300;
public:
    void func1(){
        cout << v1 << endl;
        cout << v2 << endl;
        cout << v3 << endl;
    }
};
int main(){
    Student s1;
    s1.func1();
    s1.v1 = 99;
    //s1.v2 = 11;//对象不能访问私有属性
    //s1.v3 = 33;//对象不能访问受保护的属性
    return 0;
}
```

友元类与友元函数

友元函数

在 C++ 中，类的私有成员（ private ）和只能在本类中访问，类的保护成员（ protected ）只能在本类或者子类中访问，但类的友元函数可以访问类的私有成员和保护成员。

类的友元函数是定义在类外部，但有权访问类的所有私有（ private ）成员和保护（ protected ）成员。尽管友元函数的原型有在类的定义中出现过，但是友元函数并不是成员函数。

友元函数

定义

使用 **friend** 修饰的函数，称为友元函数。

语法

```
class ClassName
{
public:
    friend void func();
};

void func()
{
}
```

使用 friend 定义了一个友元函数 func。

友元函数 案例

```
class Student
{
public:
    Student(string name, int age)
    {
        this->name = name;
        this->age = age;
    }

private:
    string name;
    int age;
};
//非成员函数
void printName(Student stu)
{
    cout << stu.name << endl; //报错
}
int main(){
    Student s1("zs",20);
    printName(s1);
    return 0;
}
```

友元函数 案例

```
class Student
{
public:
    Student(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
```

```
private:
    string name;
    int age;
```

```
};
```

//非成员函数

```
void printName(Student stu)
{
```

```
    cout << stu.name << endl; //不报错
```

```
}
```

```
int main(){
    Student s1("zs",20);
    printName(s1);
    return 0;
}
```

```
friend void printName(Student stu);
```

友元类与友元函数

友元类

在 C++ 中，可以将一个类声明为友元类，如果一个类是另一个类的友元类，友元类可以访问类的所有的私有成员。

友元类

定义

使用 **friend** 修饰的类，称为友元类。

语法

```
class ClassName
```

```
{
```

```
public:
```

```
    friend class ClassName2;
```

```
};
```

```
class ClassName2
```

```
{
```

```
};
```

使用 friend 定义了一个友元类 ClassName2，此时 ClassName2 就可以访问类的所有的私有成员

友元类

```
class Teacher;
class Student{
private:
    string name ="王五";
    int age;

public:
    void stu_print(Teacher &t);
};
class Teacher{
private:
    string name="张三";
    int age;

public:
    friend class Student;
};
void Student::stu_print(Teacher &t){
    cout << this->name <<endl;
    cout << t.name <<endl;
}
int main(){
    Student s1;
    Teacher t1;
    s1.stu_print(t1);
    return 0;
}
```

运算符重载

C++ 允许在同一作用域中的某个函数和运算符指定多个定义，分别称为函数重载和运算符重载。

重载声明是指一个与之前已经在该作用域内声明过的函数或方法具有相同名称的声明，但是它们的参数列表和定义（实现）不相同。

当调用一个重载函数或重载运算符时，编译器通过把所使用的参数类型与定义中的参数类型进行比较，决定选用最合适的定义。选择最合适的重载函数或重载运算符的过程，称为重载决策。

函数重载

```
class PrintFunc
{
public:
    void print(int i) {
        cout << i << endl;
    }

    void print(double f) {
        cout << f << endl;
    }

    void print(char c[]) {
        cout << c << endl;
    }
};
```



类里面
跟普通函数重载类似

运算符重载

```
class Student{
private:
    string name;
    int age;
public:
    Student(){}
    Student(string name,int age){
        this->name = name;
        this->age = age;
    }
};

int main(){
    Student s1("aaa",11),s2("bbb",22);
    if(s1 > s2){
        cout << "s1 > s2" <<endl;
    }
    return 0;
}
```



代码这样写有问题吗？

运算符重载

运算符重载的**本质为函数重载**，但有一定的规则需要遵循。

- 1.重载运算符时,运算符的运算顺序和优先级不变,操作数个数不变。
- 2.不能创造新的运算符,只能重载C++中已有的运算符,并且规定有6个运算符不能重载,如表所示。
- 3.运算符重载是针对新类型的实际需求,对原有的运算符进行适当的改造。一般来讲,重载后的运算符的功能应当与运算符的实际意义相符。

运算符重载

不能重载的运算符	含义
.	类属关系运算符
.*	成员指针运算符
::	作用域运算符
?:	条件运算符
#	编译预处理符号
sizeof	取数据类型的长度

运算符重载

运算符重载形式有两种，重载为类的成员函数和重载为类的友元函数。

运算符重载为类的成员函数的一般语法形式为：

函数类型 operator 运算符（形参表）

```
{  
    函数体；  
}
```

运算符重载为类的友元函数的一般语法形式为：

friend 函数类型 operator 运算符（形参表）

```
{  
    函数体；  
}
```

运算符重载

由于运算符种类与数量较多，以以下4大类中的一些符号来讲解。

4大类：

- 1.赋值运算符的重载
- 2.算术运算符的重载
- 3.关系运算符的重载
- 4.其他较为特殊的重载

赋值运算符的重载

```
class Student{
private:
    string name;
    int age;
public:
    Student(){
        name = "zs";
        age = 20;
    }
    Student(string name,int age){
        this->name = name;
        this->age = age;
    }
    void operator=(const Student &t){
        this->name = t.name;
        this->age = t.age;
    }
    void printInfo(){
        cout << this->name << " " << this->age << endl;
    }
};

int main(){
    Student s1,s2("王五",30);
    s2.printInfo();
    s2 = s1; //使用赋值运算符
    s2.printInfo();
    return 0;
}
```

算术运算符的重载

```
class complex{
public:
    complex(){
    }
    complex(double r,double i){
        this->r = r;
        this->i = i;
    }
    complex operator+(complex &c){
        complex c2;
        c2.r = this->r + c.r;
        c2.i = this->i + c.i;
        return c2;
    }
    complex operator-(complex &c){
        complex c2;
        c2.r = this->r - c.r;
        c2.i = this->i - c.i;
        return c2;
    }
    void print(){
        cout << this->r <<" " << this->i <<endl;
    }
private:
    double r,i;
};
```

```
int main(){
    complex c1(11,12),c2(10,11),c3,c4;
    c3 = c1 + c2;
    c3.print();
    c4 = c1 - c2;
    c4.print();
    return 0;
}
```

关系运算符的重载

```
class Student{
private:
    string name;
    int age;
public:
    Student(){}
    Student(string name,int age){
        this->name = name;
        this->age = age;
    }

    bool operator>(Student &t){
        if(this->name > t.name){
            return true;
        }else{
            return false;
        }
    }
};

int main(){
    Student s1("aaa",11),s2("bbb",22);
    bool b = s2 > s1; //bool b = s1 < s2;  bool b = s2 > s1;
    cout << b << endl;
    return 0;
}
```

其他较为特殊的重载

```
const int SIZE = 5;

class Safearr{
private:
    int arr[SIZE];
public:
    Safearr(){
        register int i = 0;
        for(;i<SIZE;i++){
            arr[i] = i;
        }
    }
    int operator[](int i){
        if(i >=SIZE){
            cout << "下标超过最大值" <<endl;
            return -1;
        }
        return arr[i];
    }
};

int main(){
    Safearr arr;
    cout << arr[1] <<endl;
    cout << arr[100] <<endl;
    return 0;
}
```

练习

利用友元函数，重载动物类年龄大小的代码。

代码

```
class Animal{
private:
    string name;
    int age;
public:
    Animal(){}
    Animal(string name,int age){
        this->name = name;
        this->age = age;
    }
    friend bool operator>(Animal &a1,Animal &a2);
    friend bool operator<(Animal &a1,Animal &a2);
    friend bool operator==(Animal &a1,Animal &a2);
};

bool operator>(Animal &a1,Animal &a2){
    if(a1.age >a2.age){
        return true;
    }
    else{
        return false;
    }
}

bool operator<(Animal &a1,Animal &a2){
    if(a1.age < a2.age){
        return true;
    }
    else{
        return false;
    }
}

bool operator==(Animal &a1,Animal &a2){
    if(a1.age == a2.age){
        return true;
    }
    else{
        return false;
    }
}
```

```
int main(){
    Animal a1("cat",22),a2("dog",22);
    bool b = a1 > a2;
    cout << b << endl;
    return 0;
}
```

知识点总结

1. 修饰符

重点

2. 友元类与友元函数

重点

3. 运算符重载

重点

作业

1. 创建一个分数类，重载分数加减乘除。

下节课预告

继承和多态